- **概念：基于角色的权限控制**

基于角色的权限控制（Role-Based Access Control，RBAC）是一种安全机制，用于管理用户对计算机系统资源的访问权限。RBAC 通过定义用户角色，并将这些角色与权限关联，来简化权限管理和增强安全性。以下是 RBAC 的基本概念和实现步骤：

基本概念

1. **用户（User）**：指需要访问系统资源的个体。
2. **角色（Role）**：代表一组权限的抽象，通常对应一个职务或职责。
3. **权限（Permission）**：对资源的具体操作授权，如读、写、删除等。
4. **资源（Resource）**：系统中的受保护对象，如文件、数据库表、API 接口等。

实现步骤

1. **定义角色**：
   - 确定系统中需要哪些角色，如管理员、编辑、普通用户等。
2. **定义权限**：
   - 列出系统中的所有资源及其可执行的操作。
   - 为每个角色分配相应的权限。
3. **创建用户角色映射**：
   - 将用户分配到相应的角色。
4. **权限验证**：
   - 在用户访问资源时，系统检查用户的角色是否具备相应的权限。

- **基本思路**
  - 将所有权限存储到后端
  - 将权限分配给角色，再将角色分配给用户
  - 页面初始化时，先获取用户的权限，然后根据用户的权限确定其可以访问的页面

- **实现**
  - 页面权限控制
    - 初始路由
      - 步骤
        - 将页面路由分成两部分：权限页面路由、非权限页面路由
        - 路由对象初始化时，只注册非权限路由
      - 示例代码

```
const routes: RouteRecordRaw[] = [
  {
    path: '/login',
    name: 'login',
    component: () => import('@/views/login/login.vue')
  },
  {
    path: '/:pathMatch(.*)*',
    component: () => import('@/views/not-found/not-found.vue')
  }
]

export const getDynamicRoutes: () => RouteRecordRaw[] = () => [
  {
    path: '/',
    component: Layout,
    meta: {
      pageName: '基础页'
    },
    children: [
      {
        path: 'home',
        name: 'home',
        component: () => import('@/views/home/home.vue'),
        meta: {
          pageName: '首页'
        }
      }
    ]
  }
  // ...其他页面
]

const router = createRouter({
  history: createWebHistory(CONFIG.basePath),
  routes
})
```

- userStore
  - 步骤

    - 声明用户权限信息
    - 声明根据用户权限信息递归过滤后的权限页面路由
  - 示例代码

```
type IState = {
  userinfo: IPickResponse<typeof getUserinfoApi> | null
}
type IGetters = {}
type IActions = {
  getAuthorityRoutesList(dynamicRoutes?: RouteRecordRaw[], basePath?: string): RouteRecordRaw[]
}

export const useUserStore = defineStore<string, IState, IGetters, IActions>('user', {
  state: () => ({
    userinfo: null
  }),
  getters: {},
  actions: {
    getAuthorityRoutesList(dynamicRoutes: RouteRecordRaw[] = getDynamicRoutes(), basePath = '/') {
      const result: RouteRecordRaw[] = []
      dynamicRoutes.forEach(route => {
        const routePath = path.resolve(basePath, route.path)
        if (route.children) {
          route.children = this.getAuthorityRoutesList(route.children, routePath)
          if (route.children?.length) {
            route.redirect = path.resolve(basePath, route.children[0].path)
          }
        }
        if (
          !CONFIG.authorityRoute ||
          route.children?.length ||
          this.userinfo?.authorityPathsList?.includes?.(routePath)
        ) {
          result.push(route)
        }
      })
      return result
    }
```

```
  }
})
```

- 路由守卫
  - 步骤
    - 判断是否获取了用户的权限信息，未获取则同步获取
    - 将过滤后的权限页面路由注册到路由对象
    - 跳转到登录页时，清除用户权限信息、清除已注册的权限页面路由
  - 示例代码

```
let removeRouteList: ReturnType<typeof router.addRoute>[] = []

router.beforeEach(async to => {
  const userStore = useUserStore()
  if (getToken()) {
    if (!userStore.userinfo) {
      try {
        const { data: res } = await getUserinfoApi()
        userStore.userinfo = res
        userStore
          .getAuthorityRoutesList()
          .forEach(route => removeRouteList.push(router.addRoute(route)))
        return to.path
      } catch (error: any) {
        console.error(error)
        removeToken()
        return `/login${to.path === '/' ? '' : `?redirect=${to.path}`}`
      }
    }
    if (to.path === '/login') {
      return '/'
    } else {
      return true
    }
  } else {
    if (to.path === '/login') {
      userStore.userinfo = null
      removeRouteList.forEach(item => item())
      removeRouteList = []
      return true
```

```
    } else {
      return `/login${to.path === '/' ? '' : `?redirect=${to.path}`}`
    }
  }
})
```

- 动态导航
  - 在过滤后的权限页面路由中，找到目标路由数组，循环生成导航

```
export const getRoutesList = (): {
  label: string
  routeName: string
}[] =>
  useUserStore()
    .getAuthorityRoutesList()
    .find(route => route.path === '/')
    ?.children?.map(route2 => ({
      label: route2.meta?.pageName ?? '',
      routeName: (route2.name ?? '') as string
    })) ?? []
```

- 一键同步权限
  - 说明
    - 需要将全部权限存储到后端，以用来在其他页面编辑角色权限
    - 存储的权限至少包含以下信息：权限名称、权限层级关系、相同层级权限的顺序
  - 步骤
    - 获取后端存储的权限
    - 获取前端全部权限
    - 删掉后端多出的权限
    - 新增前端多出的权限
  - 示例代码

```
const syncAuthority = async () => {
  try {
    const authorityList = (await getAuthorityListApi()).data
    const wholeAuthorityList = getWholeAuthorityList()
    const discardAuthorityList = authorityList
      .filter(
        item =>
```

```
8           !wholeAuthorityList.find(
9             item2 =>
10              item2.path === item.path &&
11              item2.pageName === item.pageName &&
12              item2.order === item.order
13          )
14      )
15      .map(item => item.id)
16    const newAuthorityList = wholeAuthorityList.filter(
17      item =>
18        !authorityList.find(
19          item2 =>
20            item2.path === item.path &&
21            item2.pageName === item.pageName &&
22            item2.order === item.order
23        )
24    )
25    await deleteAuthority(discardAuthorityList)
26    await addAuthority(newAuthorityList)
27    ElMessage.success('同步成功！')
28  } catch (error: any) {
29    ElMessage.error(error.msg || '同步失败！')
30    console.error(error)
31  }
32 }
33
34 const getWholeAuthorityList = (
35   dynamicRoutes: RouteRecordRaw[] = getDynamicRoutes(),
36   basePath: string = ''
37 ): Parameters<typeof addAuthorityApi>[0][] => {
38   const result: Parameters<typeof addAuthorityApi>[0][] = []
39   dynamicRoutes.forEach((route, index) => {
40     const routePath = path.resolve(basePath, route.path)
41     result.push({
42       path: routePath,
43       pageName: route.meta?.pageName ?? `未命名：${routePath}`,
44       order: index + 1
45     })
46     if (route.children?.length) {
47       result.push(...getWholeAuthorityList(route.children, routePath))
48     }
49   })
```

```
  return result
}

const deleteAuthority = async (idList: string[]) => {
  idList.length && (await deleteAuthorityApi(idList))
}

const addAuthority = async (dataList: Parameters<typeof addAuthorityApi>[0][]) => {
  const proList: Promise<unknown>[] = []
  dataList.length && dataList.forEach(item => proList.push(addAuthorityApi(item)))
  await Promise.all(proList)
}
```

- 角色权限编辑
  - 获取后端存储的权限，生成复选框树
  - 编辑并保存

- 其他问题
  - 不能缓存最外层的权限页面，因为每次重新使用addRoute注册路由后，即使路由相同，也不会再复用之前的组件，会出现两个组件实例。